

An Implementation of Attribution-by-Design for Ethical AI-Generated Music

Logan Costello
Computer Science
Cal Poly, SLO

James Irwin
Computer Science
Cal Poly, SLO

Charlie Jansen
Computer Science
Cal Poly, SLO

Abstract—Sound Ethics is an organization that develops and advocates for ethical artificial intelligence (AI). Their aim is to ensure that musicians and artists are able to thrive in music careers alongside the development of generative music models [6]. At Cal Poly, we have partnered with Sound Ethics to implement an AI framework that generates music and gives proper attribution to the original artists. This implementation, based on the Sony AI Attribution-by-Design architecture, can be used to ethically generate new music and enables developers to give proper credit and compensation to the original artists [1]. Additionally, our implementation allows for multiple conditioning audios, a feature which very few other open-source music generation models natively support.

I. INTRODUCTION

The current problem with AI-generated music is that existing frameworks do not give proper attribution to the original artists. That is, these models do not give credit to the music that they use to generate outputs. AI models do not generate music out of thin air. Rather, they use a wide variety of musical references, which are frequently obtained and used without the original artist’s consent, to create coherent and enjoyable pieces of music. This process poses an ethical dilemma: while AI models are able to generate good music, existing approaches fail to give proper credit to the original art (and thus artists) that were used to generate this music in the first place [1]. Therefore, original artists are being left uncredited, unpaid, and unable to track what is being generated based on their work.

When using existing music to create new music, artists must credit the original work via long-established legal protocols [1]. For example, in order for a song to be sampled, the original artist must consent to the use of their song, be credited in the new song, and be financially compensated if the new song is profitable. Current laws naturally fit in the context of AI-generated music: if the generated song is based on an existing song, the artist of the existing song must consent, be credited, and be compensated. However, there currently does not exist any protocols for demonstrating or proving that a piece of AI-generated music is based on a specific piece of existing music.

The purpose of this project is to implement an application that allows users to generate music while giving proper credit to artists’ original work. We have based our approach on the proposed solution of the Sony AI Team, who have outlined an architecture that can be used to give attribution while

generating music [1]. Our implementation does not focus on legally proving attribution. Instead, our application is designed to provide users with choices that allow for attribution to be tracked at the time of generation and used to prove attribution in the future.

II. THE SONY PAPER

The Sony AI paper **Attribution-By-Design: Ensuring Inference-Time Provenance in Generative Music Systems** is the backbone of our approach [1]. In this paper, the authors outline the problem, which is that existing generative music architectures do not give proper credit to original artists. To fix this problem, they propose a framework in which generated music is conditioned on specific songs. That is, the application generates music on the basis of specific, user-selected reference tracks. These reference songs are then given credit for being used to make the new music. The paradigm behind this framework is that user choice is essential to proper attribution. According to the Sony AI Team, explicit user choices are required to give credit to reference material. Therefore, their solution allows the user to choose audio clips that directly influence the generated output. As a result, they can give appropriate credit to the selected reference tracks.

A. Training-Time Attribution vs Inference-Time Attribution

One important aspect of this paper is the distinction between training-time attribution (TTA) and inference-time attribution (ITA). The authors propose that these are “ontologically distinct” concepts and can therefore be addressed independently [1]. TTA consists of all the music that is used to initially train the model, and generally encompasses a wide swath of music. ITA, on the other hand, consists of the songs that are used to condition the model when generating a specific output, and encompasses only a few songs. TTA can be thought of as crediting a song for how the model is constructed, while ITA can be thought of as crediting a song for how a specific output is created.

The authors’ proposed framework involves ITA and not TTA. That is, their solution involves giving credit to the specific references that are used to condition the model and generate an output, not the large catalog of music that was used to initially train the model. The authors expect solutions to TTA to be complementary to their architecture, but believe

that addressing ITA is both easier and more important than addressing TTA [1].

B. Attribution-by-design Framework

The Attribution-By-Design framework properly tracks attribution from the start of the generation process, rather than after it. This framework relies on two important factors: many opportunities for conditioning and explicit user selection.

For the former, users must be able to condition the model to produce specific outputs, otherwise there is no inference-time attribution. An example of model conditioning is selecting an existing song to inspire a specific part of the generated output, such as the drums. For instance, a user may generate a song with drums that sound like those in “Sexyback” by Justin Timberlake and Timbaland. The user may do so by uploading an audio sample. It is important that there are many ways to condition the model. More conditioning methods not only make the tool more useful to the user, but also create more opportunities for explicit, trackable choices. These choices are then used when assigning attribution.

For the latter, it is important that the user makes explicit choices within the framework, rather than the application making decisions on their behalf. Continuing with our drum conditioning example, the user choosing to use the drums from “Sexyback” to influence the generated output is analogous to an artist choosing to sample the same song. Like sampling, choosing a song is an explicit and provable choice, so it seems reasonable that making this choice entails the same legal protocols as sampling. Any choice made algorithmically weakens the proof that a given song was used intentionally and therefore also weakens the attribution for the original song’s artist.

To follow this Attribution-By-Design framework, our application creates many opportunities for explicit user choices about the generated output. We have also added opportunities for user control that were not specified by Sony AI, such as tempo and duration, to allow for maximum user creativity. Any time the user makes a choice, our application tracks it, ensuring that attribution is properly assigned.

III. APPLICATION OVERVIEW

In this section, we expand on our application and review our system architecture. As discussed in the previous section, it is integral that our application generates output on specific user input to give proper attribution. Therefore, there are several methods of input. One such method is audio upload, in which the user uploads their own audio clip and selects what component of the track they want to condition on. These components can be the drums, bass, piano, guitar, or melody of each provided audio clip. The user may choose to upload up to one audio per conditioning component. Additionally, the user is able to provide a textual description of their desired output. For example, the user may use the description “jazzy, uplifting bossa nova track” to create a happy, bossa nova-style song. Finally, the user may control key, beats per minute (BPM), duration, and several other model generation hyperparameters.

A. Architecture

Our application relies on various open-source tools to handle user input and generate an output. The basis of our application is ACE-Step, which is a text-to-music generation model that creates output based on a textual description and a single conditioning audio clip [13]. Before generating with ACE-Step, we use two other tools to preprocess the audio clips uploaded by the user. The first tool is Facebook’s Demucs, which is a stem-splitter [5]. Stem-splitters can be used to isolate specific sounds from a given audio. In our application, we use Demucs to isolate the drums, bass, piano, guitar, and melody from the user uploaded audio clips. Since the user can upload multiple audios but ACE-Step only accepts a single audio, we use a python package named librosa to combine all of the isolated audio clips [4]. This combined audio is then used as the conditioning audio in the generation process. The following is a diagram of our entire system architecture:

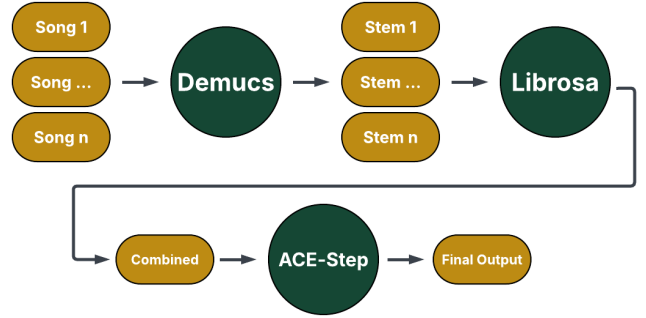


Fig. 1. A diagram of the entire system architecture

The purpose of our architecture is to generate output based on multiple audio inputs. As we have found through our research, a massive limitation of current generative music models is that they are only designed to use one audio as input. This single-input design severely limits user choice and, therefore, opportunities for attribution. Within our framework, users can upload and select components from up to five different audio clips. This design increases the number of user choices and creates more opportunities for attribution: we can give credit to five songs instead of one. Additionally, the user has more control in how the output audio will sound. Instead of being able to use only one song as a reference, a user can now specify different audio components from several audios. For example, they could specify that the output should have drums like “Sexyback” and piano like “Rocket Man” by Elton John. This distinction is integral to our work. While other models only allow for a single reference audio, our framework allows users to upload multiple audios when generating an output. This improvement allows for more user choices, creates more opportunities for attribution, and gives users more creative control.

B. Frontend

The purpose of the frontend is to allow users to easily create a specific kind of song. The following is an image of our implementation, which is styled according to Sound Ethics’s branding:

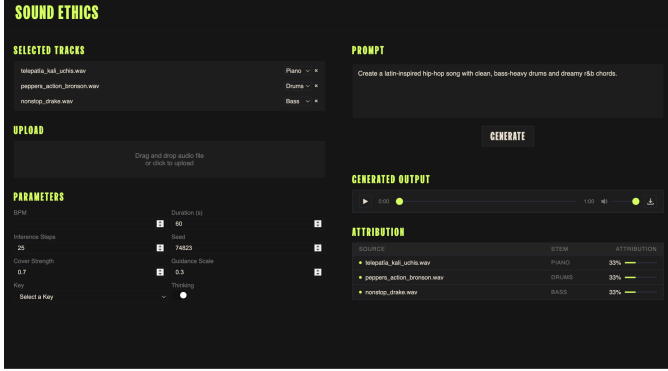


Fig. 2. An image of our current implementation

The top right of our website is where the user may input a prompt and generate a new song. This functionality is standard for all music generation models. On the left side, the user may upload multiple conditioning audios and select which musical component they would like to use to condition the output on. Additionally, they may adjust the hyperparameters. Once the user is satisfied with their inputs, they may click “Generate.” After 10-30 seconds, our backend generates an output and the frontend displays it to the user on the bottom right. The user can play and download the generated output. Additionally, the frontend displays attribution to the user as a table. Each row of the table is an input audio that was used during generation. The song name, the stem used, and the percentage of attribution given to the song are all displayed and tracked.

IV. EVALUATION

Having explained the details of our application, we now review its evaluation. Since music quality cannot be measured objectively, we conducted a survey comparing songs generated by our framework with those generated by the baseline ACE-Step model. Participants assessed four aspects of quality:

- 1) Sound Quality: The overall quality of the sound of the output
- 2) Prompt Similarity: How much it seems like the prompt influenced the output
- 3) Input Similarity: How much it seems like the input songs influenced the output
- 4) Cohesiveness: How well the individual parts of the song feel like they fit together.

We selected these metrics to answer two questions:

- 1) Does our framework’s output more closely align with user inputs?
- 2) Is our framework’s output better?

To answer the first of these questions, we would be looking for high scores for Prompt Similarity and Input Similarity,

since those are the aspects of quality that relate to how the output aligns with the given inputs. For the second question, we would need scores for Sound Quality and Cohesiveness, because those aspects relate to the general quality of the output rather than how well it matches the inputs. With these qualities and questions, it was time for us to create a survey to answer them.

Our survey consisted of nine generated outputs. Five were generated by our implementation and four by ACE-Step as a baseline. The nine outputs were randomly shuffled and participants were not told which model generated each song. For the first five audio outputs, participants were provided with the input prompt and audio clips that were used to generate that output. Then, they were asked to assess the output across all four metrics on a scale of one to five. For the last four outputs, participants were only given the generated output and were asked to rate just Audio Quality and Cohesiveness. We surveyed fourteen participants, including classmates from our data science capstone class and people associated with Sound Ethics (employees, partners, etc). Participant ratings were all anonymous. We averaged all responses to compare our implementation (labeled “Sound Ethics”) to baseline ACE-Step. The following figure displays these averages:

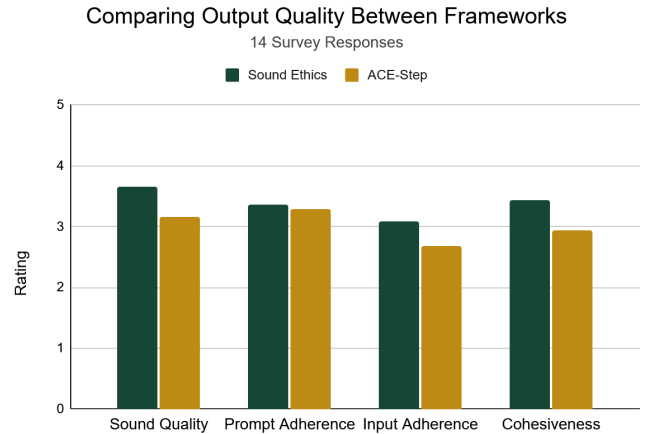


Fig. 3. The results from our survey, comparing our architecture to ACE-Step

As shown in the figure, our implementation outperforms baseline ACE-Step across all four metrics. Therefore, we have answered both of our research questions:

- 1) Does our framework’s output more closely align with user inputs?
 - a) Yes. As shown in the results for Prompt Adherence and Input Adherence, our framework’s output is more similar to both the user-supplied text prompt and the audio clips.
- 2) Is our framework’s output better?
 - a) Yes. As shown in the results for Cohesiveness and Sound Quality, our framework’s output is more cohesive and higher quality.

Our results indicate that allowing users to condition on multiple input audios increases creative control and overall output quality. It is worth noting that very few open-source models natively support conditioning on different parts of multiple input audios. The success of our implementation suggests that this functionality can be achieved without training a new music generation model designed for this specific task, all while improving upon modern quality standards.

V. OTHER WORK

This project took a significant amount of research, experimentation, and testing, some of which was not used in our final implementation. This section summarizes that work.

We started our process by researching artificial music generation and running experiments. While this research did not directly contribute to our project deliverables, it was instrumental in helping us understand the complex subject of music generation models. We read a variety of academic papers and project implementations while conducting this research, including CLAP [7], MusicGen [2], Retrieval-Augmented Text to Audio Generation [8], MuseControlLite [9], HeartMuLa [10], Songbloom [11], and Magenta [12]. By researching these music generation models, text-to-audio models, and song similarity algorithms, we were able to better understand the music generation landscape and make informed decisions about what technologies we could incorporate in our project.

Additionally, in order to further understand how these models condition on specific inputs, we tried modifying the MusicGen attention layer to see if we could successfully condition on inputs the model wasn't designed for. We successfully modified the model and we were able to show that adjusting the values in the attention layer resulted in different outputs. However, we were not able to change the model in a way that resulted in quality outputs. This experiment helped us further understand model conditioning. More importantly, it demonstrated that it was not feasible for us to create our own music generation models. We realized that, going forward, we would need to leverage existing open-source models.

VI. FUTURE WORK

While we are satisfied with our final product, there are several areas for improvement. If another team was to continue our work, we would suggest the following.

First, the team should implement a proper attribution method. Currently, we assign equal amounts of attribution to each song, but it's unlikely that each input audio equally influences the output. Additionally, because the user provides a prompt and other parameters such as tempo and key, it seems reasonable for some of the attribution to be given to the user. Our understanding is that other implementations utilize audio similarity algorithms. Adding that functionality to our framework would make our attribution process much more robust.

Second, users should be able to select existing songs from a database in addition to uploading them manually. This feature would not only improve the user experience, but also allow for

more artist protections. For instance, the database would only consist of music that artists explicitly allowed our application to use. They could even specify which stems of each song could be used [1].

Third, output should be "watermarked" so that attribution cannot be ignored. For example, output files could have non-removable attribution data embedded within them. An established protocol for this procedure is the Coalition for Content Provenance and Authenticity's (C2PA) Content Credentials [14]. The team could implement Content Credentials to ensure that attribution data is stored in output files.

Finally, we suspect that a model specifically trained to generate music conditioned on multiple songs (or even multiple stems) would produce better quality outputs than ours. We were frustrated that very few existing models do so. During our development, we tried using Facebook's JASCO model [5], which took drum and melody stems as inputs. However, we decided against using JASCO because its output was low-quality and the number of stems was limited. Designing a model that uses multiple stems as input would eliminate the need for our preprocessing and could generate better results.

In addition to these tasks, there are several questions that our solution does not answer. For example, we have repeatedly mentioned that our application determines attribution by tracking the user's selections, but it is not immediately obvious how these selections are turned into compensation percentages. For example, if the user conditions the output on two different songs, should both of those songs be equally attributed and compensated? If not, how do you determine which song contributed more to the final output? Additionally, consider two scenarios: one where the user conditions on a single song with high model temperature and one where the user conditions on a single song with low temperature. When model temperature is high, outputs are more diverse and therefore less similar to the inputs. Given this, should a song used with high model temperature have less attribution and compensation than a song used with low temperature? Once again, the answer is not obvious. These questions are important to address, but we determine that they are philosophical and legal in nature as opposed to engineering questions.

REFERENCES

- [1] Fabio Morreale, Wiebke Hutiri, Joan Serrà, Alice Xiang, and Yuki Mitsufuji. (2025). Attribution-By-Design: Ensuring Inference-Time Provenance in Generative Music Systems. arXiv. <https://doi.org/10.48550/arXiv.2510.08062>
- [2] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. (2023). Simple and Controllable Music Generation. arXiv. <https://doi.org/10.48550/arXiv.2306.05284>
- [3] Jong Wook Kim, Justin Salamon, Peter Li, Juan Pablo Bello. (2018). Crepe: A Convolutional Representation for Pitch Estimation. arXiv. <https://doi.org/10.48550/arXiv.1802.06182>
- [4] Brian McFee, Colin Raffel, Dawen Liang, Daniel P.W. Ellis, Matt McVicar, Eric Battenberg, Oriol Nieto. (2015). librosa: Audio and Music Signal Analysis in Python. In Proceedings of The 14th Python In Science Conf, pp. 18-25. <https://proceedings.scipy.org/articles/Majora-7b98e3ed-003.pdf>
- [5] Or Tal, Alon Ziv, Itai Gat, Felix Kreuk, Yossi Adi. (2024). Joint Audio And Symbolic Conditioning for Temporally Controlled Text-To-Music Generation. arXiv. <https://doi.org/10.48550/arXiv.2406.10970>

- [6] Sound Ethics. <https://www.soundethics.org/>
- [7] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Marianna Nezhurina, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. (2024). Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation. arXiv. <https://doi.org/10.48550/arXiv.2211.06687>
- [8] Yi Yuan, Haohe Liu, Xubo Liu, Qiushi Huang, Mark D. Plumbley, and Wenwu Wang. (2024). Retrieval-Augmented Text-to-Audio Generation. arXiv. <https://doi.org/10.48550/arXiv.2309.08051>
- [9] Fang-Duo Tsai, Shih-Lun Wu, Weijaw Lee, Sheng-Ping Yang, Bo-Rui Chen, Hao-Chung Cheng, and Yi-Hsuan Yang. (2024). MuseControlLite: Multifunctional Music Generation with Lightweight Conditioners. arXiv. <https://doi.org/10.48550/arXiv.2506.18729>
- [10] Dongchao Yang, Yuxin Xie, Yuguo Yin, Zheyu Wang, Xiaoyu Yi, Gongxi Zhu, Xiaolong Weng, Zihan Xiong, Yingzhe Ma, Dading Cong, Jingliang Liu, Zihang Huang, Jinghan Ru, Rongjie Huang, Haoran Wan, Peixu Wang, Kuoxi Yu, Helin Wang, Liming Liang, Xianwei Zhuang, Yuanyuan Wang, Dingdong Wang, Haohan Guo, Junjie Cao, Zeqian Ju, Songxiang Liu, Yuewen Cao, Heming Weng, and Yuexian Zou. (2026). HeartMuLa: A Family of Open Sourced Music Foundation Models. arXiv. <https://doi.org/10.48550/arXiv.2601.10547>
- [11] Chenyu Yang, Shuai Wang, Hangting Chen, Wei Tan, Jianwei Yu, Haizhou Li. (2025). SongBloom: Coherent Song Generation via Interleaved Autoregressive Sketching and Diffusion Refinement. arXiv. <https://doi.org/10.48550/arXiv.2506.07634>
- [12] Magenta. Google. <https://magenta.withgoogle.com/>
- [13] Junmin Gong, Sean Zhao, Sen Wang, Shengyuan Xu, Joe Guo. (2025). ACE-Step: A Step Towards Music Generation Foundation Model. arXiv. <https://doi.org/10.48550/arXiv.2506.00045>
- [14] C2PA. Coalition for Content Provenance and Authority. <https://c2pa.org/>